

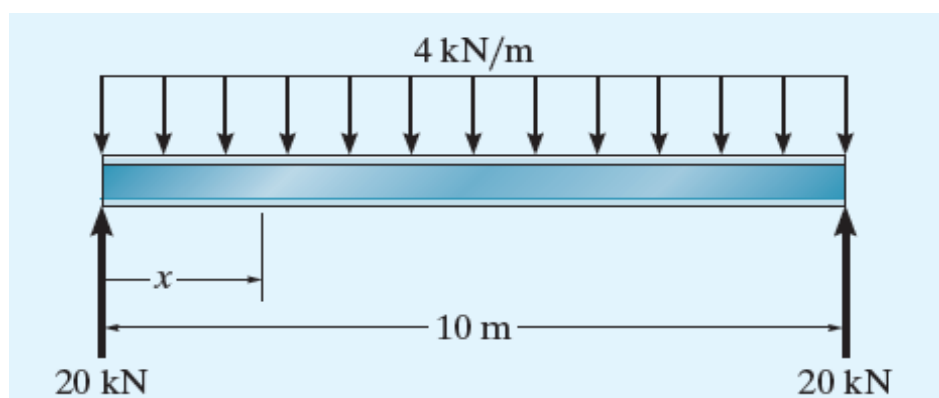
PROJECT 2: SOLVING SECOND ORDER DIFFERENTIAL EQUATION OF DEFLECTION OF SIMPLY SUPPORTED BEAM UNDER UNIFORMLY DISTRIBUTED LOAD WITH PINN:

ABSTRACT:

This work presents a development of Physics-Informed Neural Network (PINN) approach to solve the beam deflection problem governed by a second-order ordinary differential equation derived from Euler-Bernoulli beam theory. The governing physics is applied directly into the neural network loss function using automatic differentiation in PyTorch without any labelled training data. Both Dirichlet and Neumann boundary conditions are enforced simultaneously through a weighted loss formulation. The trained PINN is validated with double integration method analytical solution and the model successfully demonstrates excellent agreement with actual physics. This approach highlights the potential of PINNs as a meshless alternative to classical numerical methods such as the Finite Element Method for structural analysis problems.

PROBLEM STATEMENT:

The problem statement is a simple supported beam, hinged at one end and roller-supported on other end subjected to a downward uniformly distributed load with constant flexural rigidity throughout the whole beam span.



Engineering data:

- Beam length(L): 10 m
- UDL(w) = 4 kN/m
- Support condition: Simply Supported
- Flexural Rigidity (EI): $6.25 \times 10^5 \text{ kN.m}^2$

Boundary Condition:

- Deflection at x=0 ($u(0)$) = 0
- Deflection at x=10 ($u(10)$) = 0
- Slope at x=5 ($u'(5)$) = 0
- Slope at x = 0 ($u'(0)$) = $\frac{wL^3}{24EI}$
- Slope at x = 10 ($u'(10)$) = $-\frac{wL^3}{24EI}$

ANALYTICAL SOLUTION:**Governing Equation:**

$$EI \frac{d^2y}{dx^2} = M$$

$$\frac{d^2y}{dx^2} - \frac{M}{EI} = 0$$

Deflection = y

$$\text{Slope} = \frac{dy}{dx}$$

Bending Moment = M

Solution:

Bending Moment at span x from left end of beam,

$$M = \frac{wL}{2}x - \frac{w}{2}x^2$$

Putting the values,

$$M = 20x - 2x^2$$

Solving the equation,

$$\frac{d^2y}{dx^2} - \frac{M}{EI} = 0$$
$$\frac{d^2y}{dx^2} - \frac{20x - 2x^2}{EI} = 0$$

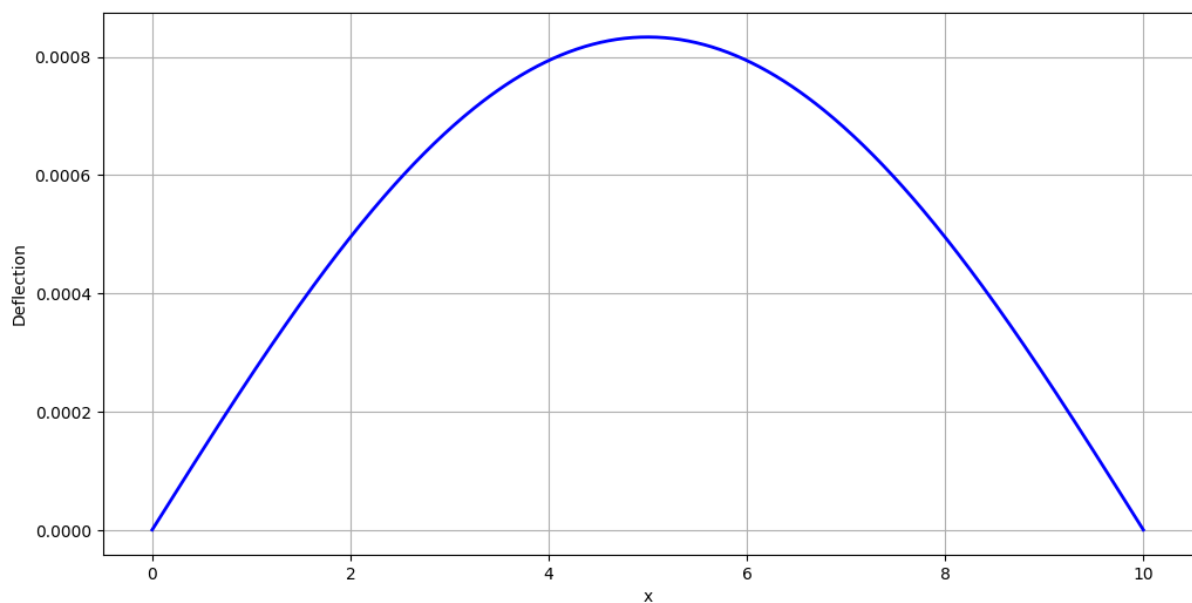
After applying boundary conditions,

Deflection at $x=0$ ($u(0)$) = 0

Deflection at $x=10$ ($u(10)$) = 0

Slope at $x=5$ ($u'(5)$) = 0

$$y = \frac{w}{24EI}x^4 - \frac{wL}{12EI}x^3 + \frac{wL^3}{24EI}x$$

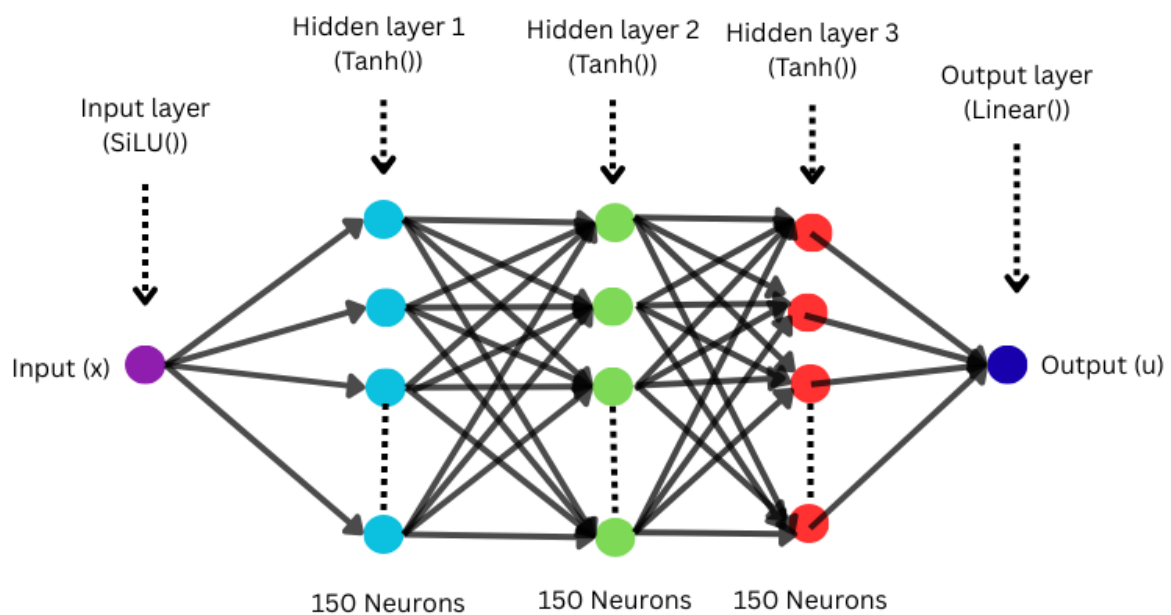


Analytical Curve of Deflection vs Span

PINN METHODOLOGY:

Network Architecture

- Input Layer:
 - I. Number of Neurons: 1
 - II. Activation Function: SiLU()
- Hidden Layer 1, 2, 3:
 - I. Number of Neurons in each layer: 150
 - II. Activation Function in each Layers: Tanh()
- Output Layer:
 - I. Number of Neurons: 1
 - II. Activation Function: Linear()



PINN Network Architecture Diagram

Mixed Activation Function Network:

In this PINN network, first layer is activated by SiLU activation which is used for its non-saturating property. It will keep updating the gradient.

In the hidden three layers, Tanh activation is applied to obtain smooth higher order derivatives which is essential for accuracy of PINN model. This mixed activation function network practice increases the overall performance of PINN.

Loss Function:

Types of losses which are calculated

- PDE Loss: This calculates the deviation of PINN model's output from the value obtained by the actual 2nd order differential equation.
- Boundary Condition Loss: This calculates the deviation of PINN model's value from the actual boundary conditions of the 2nd order differential equation. This loss enforces the physics in the PINN.

KEY CODE SNIPPETS:

1. Network:

```
class PINN(nn.Module):
    """Fully-connected network that approximates u(x)."""

    def __init__(self, hidden_layers: int = 4, neurons: int = 150):
        super().__init__()
        layers = [nn.Linear(1, neurons), nn.SiLU()]
        for _ in range(hidden_layers - 1):
            layers += [nn.Linear(neurons, neurons), nn.Tanh()]
        layers.append(nn.Linear(neurons, 1))
        self.net = nn.Sequential(*layers)

        # Xavier initialisation for better convergence
        for m in self.net:
            if isinstance(m, nn.Linear):
                nn.init.xavier_normal_(m.weight)
                nn.init.zeros_(m.bias)

    def forward(self, x: torch.Tensor) -> torch.Tensor:
        return self.net(x)
```

2. Residual:

```
residual = u_xx + ((w * L * 0.5) * x - (w * 0.5) * (x**2)) / (625e3)      # u'' - M/EI = 0
return residual
```

3. Loss function:

```
total = loss_pde + lambda_bc * loss_bc
return total, loss_pde, loss_bc
```

4. Boundary conditions:

```
w = 4.0
L = 10.0
EI = 625e3
slope = (w * (L**3)) / (24.0 * EI)

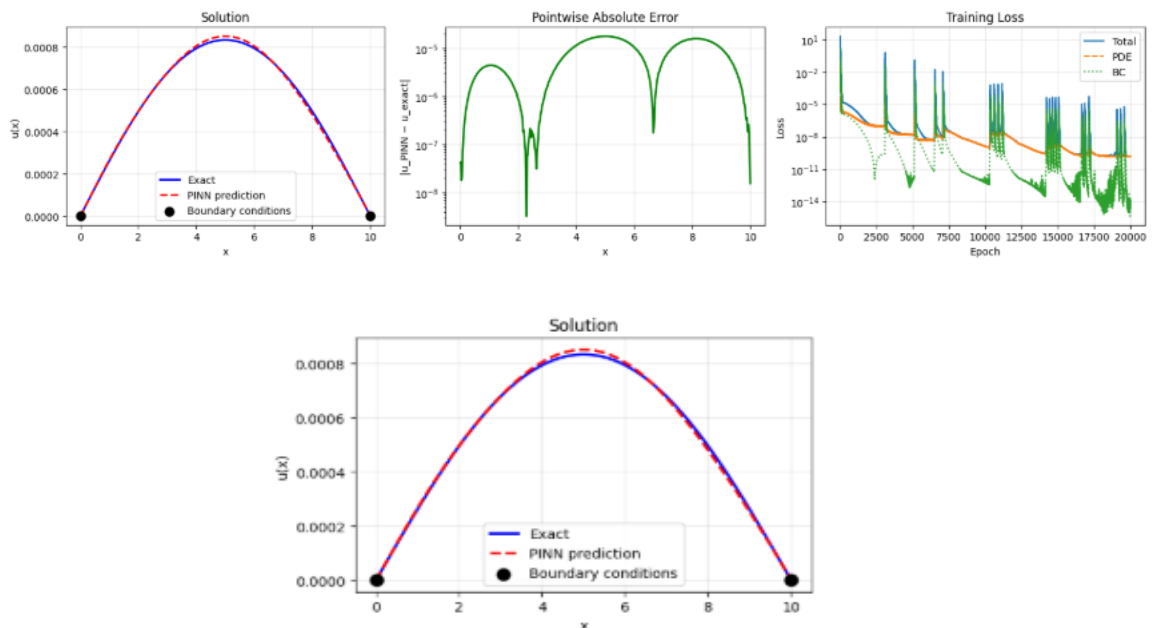
# Boundary conditions: u(0)=0, u_x(5)=0, u(10)=0, u_x(0) = (wL^3)/24EI, u_x(10) = -(wL^3)/24EI
x_bc_1 = torch.tensor([[0.0]], dtype=torch.float32).to(device)
u_bc_1 = torch.tensor([[0.0]], dtype=torch.float32).to(device)
x_bc_2 = torch.tensor([[10.0]], dtype=torch.float32).to(device)
u_bc_2 = torch.tensor([[0.0]], dtype=torch.float32).to(device)
x_bc_3 = torch.tensor([[0.0]], dtype=torch.float32).to(device)
ux_bc_3 = torch.tensor([[+slope]], dtype=torch.float32).to(device)
x_bc_4 = torch.tensor([[5.0]], dtype=torch.float32).to(device)
ux_bc_4 = torch.tensor([[0.0]], dtype=torch.float32).to(device)
x_bc_5 = torch.tensor([[10.0]], dtype=torch.float32).to(device)
ux_bc_5 = torch.tensor([[(-1) * slope]], dtype=torch.float32).to(device)
```

RESULTS:

The results are shown below. The results prominently reflect strong PINN model which accurately predicts beam deflection.

Max absolute error : 1.7721e-05
Mean absolute error: 7.6731e-06
Plot saved → pinn_results.png

PINN Solution of $u'' - M/EI = 0$, $u(0)=u(10)=0$, $u'(5) = 0$



CONCLUSION:

This study demonstrates successful application of a Physics-Informed Neural Network to solve the Euler-Bernoulli beam deflection problem governed by a second-order ODE. The PINN framework, implemented in PyTorch, directly incorporates the governing physics and boundary conditions into the training loss through automatic differentiation. This eliminates the need for labeled training data or mesh generation. The usage of hybrid activation method of SiLU and Tanh functions, simultaneous application of Dirichlet and Neumann boundary conditions have resulted a PINN model with high accuracy and maximum pointwise error of 1.77×10^{-5} m and a PDE residual loss of 1.49×10^{-10} compared to the analytical solution.